W360 Backend API

REST API behind the W360 web app: food scanning, meal logging, nutrition stats, vitals, and the AI chat assistant.

- **Base URL:** `{API_BASE}/api` (ask the team for the environment URL)
- **Format:** JSON request and response bodies (`Content-Type: application/json`)
- **Data residency:** each user's data is stored and processed in their own region. Clients don't choose the region; it's handled server-side.

Authentication

Every endpoint except `/health` and `/api/testimonials` requires a Google Sign-In ID token:

```
Authorization: Bearer <google_id_token>
```

The server identifies the user from the token's email. Clients never send a user ID. A missing, invalid or expired token returns `401`.

Errors

Errors return a JSON body with a `detail` message:

```
{ "detail": "Daily upload limit reached (1000 images/day). Try again tomorrow." }
```

Status	Meaning
400	Invalid input (bad file type, missing field, too many images)
401	Missing or invalid token
404	Resource not found
429	Rate limit or daily limit exceeded
500	Server error
502	The AI service failed; safe to retry

Rate limits are per client, per minute. Each endpoint's limit is listed below.

Endpoints at a glance

Method	Path	Purpose
GET	`/health`	Service health check
POST	`/api/upload/presign`	Get a one-time URL for uploading an image or video
POST	`/api/analyze`	Analyze uploaded food photos for nutrition
POST	`/api/meals`	Save a meal
GET	`/api/meals`	List meals
DELETE	`/api/meals/{meal_id}`	Delete a meal
GET	`/api/nutrition/summary`	Nutrition totals by day, week, month or year
POST	`/api/facescan/save`	Save a vitals scan computed on the device
GET	`/api/vitals/summary`	Vitals scan history for charts
POST	`/api/facescan/analyze`	Server-side vitals from a face video (not used by the current UI)
POST	`/api/chat`	Send a message to the AI nutrition assistant
POST	`/api/weekly-summary/generate`	Generate a weekly summary
GET	`/api/weekly-summary`	Get the latest weekly summary
GET	`/api/testimonials`	Static testimonials for the landing page

Health

`GET /health`

No auth required.

```
{ "status": "healthy" }
```

Uploads

Images and videos are uploaded in two steps: get a presigned URL from the API, then `PUT` the file straight to that URL. Uploaded bytes never pass through the API server.

`POST /api/upload/presign`

Limit: 20/min

Request

```
{
  "filename": "lunch.jpg",
  "content_type": "image/jpeg",
  "email": "user@example.com"
}
```

Allowed type	Extensions
`image/jpeg`, `image/png`, `image/gif`, `image/webp`	`.jpg` `.jpeg` `.png` `.gif` `.webp`
`video/mp4`, `video/quicktime`, `video/webm`	`.mp4` `.mov` `.webm`

Response

```
{ "url": "<presigned upload URL>", "key": "<object key>" }
```

Next step: `PUT` the raw file to `url` with the same `Content-Type` header. The URL expires after **5 minutes**. Keep `key`; you pass it to `/api/analyze` or `/api/facescan/analyze`.

Food analysis

`POST /api/analyze`

Limit: 10/min. Each user can upload up to 1,000 images per day.

Analyzes 1 to 5 uploaded food photos. Returns the estimated dish and its nutrition, and saves the scan to the user's history.

Request

```
{
  "images": [
    { "key": "<key from presign>", "content_type": "image/jpeg" }
  ]
}
```

Response (main fields)

```
{
  "description": "Grilled chicken with rice and salad",
  "calories": "520 kcal",
  "protein": "38g",
  "carbs": "55g",
  "fat": "14g",
  "fiber": "6g",
  "sugar": "4g",
  "micronutrients": { "iron": "2.1mg", "vitamin_c": "18mg" },
  "confidence": 0.86
}
```

Nutrient values come back as display strings that include units. `confidence` runs from 0 to 1.

Meals

`POST /api/meals`

Limit: 30/min

Request

```
{
  "mealName": "Lunch",
  "mealType": "lunch",
  "description": "Office lunch",
  "imageUrl": "",
  "totalNutrition": {
    "calories": 520, "protein": 38, "carbs": 55,
    "fat": 14, "fiber": 6, "sugar": 4
  },
  "dishes": [
    {
      "name": "Grilled chicken",
      "servingSize": "150g",
      "nutrition": {
        "nf_calories": 250, "nf_total_fat": 6, "nf_total_carbohydrate": 0,
        "nf_dietary_fiber": 0, "nf_sugars": 0, "nf_protein": 30
      }
    }
  ]
}
```

- `mealType` must be one of: `breakfast`, `lunch`, `dinner`, `snack`, `other`
- `mealName` and at least one dish are required.

Response

```
{ "meal_id": "20261028T120000Z_Lunch", "message": "Meal 'Lunch' saved successfully." }
```

`GET /api/meals`

Limit: 30/min

Returns the user's meals, newest first. Food scans from `/api/analyze` are included automatically.

Query param	Type	Default	Notes
`startDate`	`YYYY-MM-DD`	none	inclusive
`endDate`	`YYYY-MM-DD`	none	inclusive
`limit`	int	100	

Response

```
{
  "meals": [
    {
      "id": "20261028T120000Z_Lunch",
      "meal_name": "Lunch",
      "meal_type": "lunch",
      "description": "Office lunch",
      "image_url": "",
      "logged_at": "20261028T120000Z",
      "total_calories": 520,
      "total_protein": 38,
      "total_carbs": 55,
      "total_fat": 14,
      "total_fiber": 6,
      "total_sugar": 4,
      "meal_dishes": []
    }
  ]
}
```

`logged_at` is UTC in the format `YYYYMMDDTHHMMSSZ`.

`DELETE /api/meals/{meal_id}`

Limit: 30/min

```
{ "message": "Meal deleted successfully." }
```

Returns `404` if the meal doesn't exist. Only saved meals can be deleted, not food scans.

Nutrition stats

`GET /api/nutrition/summary`

Limit: 30/min

Adds up the user's meals and scans into time buckets.

Query param	Type	Default	Notes
`period`	`daily` | `weekly` | `monthly` | `yearly`	`daily`	bucket size
`startDate`	`YYYY-MM-DD`	none	inclusive
`endDate`	`YYYY-MM-DD`	none	inclusive
`timezone`	IANA name, e.g. `Asia/Kolkata`	`UTC`	decides which local day a meal counts toward
`limit`	int	365	maximum number of buckets

Response. The date field's name depends on `period`: `date`, `week_start` (a Monday), `month_start` or `year_start`.

```
{
  "data": [
    {
      "date": "2026-10-28",
      "total_calories": 1850.0,
      "total_protein": 80.0,
      "total_carbs": 210.0,
      "total_fat": 60.0,
      "total_fiber": 25.0,
      "total_sugar": 30.0,
      "meal_count": 3,
      "total_vitamin_a": 0.0, "total_vitamin_c": 0.0, "total_vitamin_d": 0.0,
      "total_vitamin_b12": 0.0, "total_iron": 0.0, "total_calcium": 0.0,
      "total_potassium": 0.0, "total_sodium": 0.0, "total_zinc": 0.0,
      "total_magnesium": 0.0
    }
  ]
}
```

Buckets are sorted oldest first. Days with no meals are left out.

Vitals

Vitals are wellness estimates from a face scan. They are not medical measurements, and apps should say so wherever they show them.

`POST /api/facescan/save`

Limit: 20/min

Saves a vitals scan computed on the device. The server adds the timestamp.

Request

```
{
  "heart_rate": { "status": "ok", "heart_rate_bpm": 72 },
  "bp": { "status": "ok", "sbp": 118, "dbp": 76 },
  "extras": {
    "hrv_sdnn_ms": 44,
    "breathing_rate_bpm": 14,
    "stress_index": 1.8
  },
  "source": "device"
}
```

Each metric has its own `status`. Values count only when `status` is `"ok"`.

Response

```
{ "status": "ok" }
```

`GET /api/vitals/summary`

Limit: 30/min

Returns one row per scan, oldest first, for charts.

Query param	Type	Default	Notes
`startDate`	`YYYY-MM-DD`	none	inclusive
`endDate`	`YYYY-MM-DD`	none	inclusive
`limit`	int	60	most recent N scans

Response

```
{
  "data": [
    {
      "date": "20261025T080000Z",
      "heart_rate_bpm": 72,
      "systolic": 118,
      "diastolic": 76,
      "hrv_sdnn_ms": 44,
      "breathing_rate_bpm": 14,
      "stress_index": 1.8
    }
  ]
}
```

A metric that wasn't measured comes back as `null`.

`POST /api/facescan/analyze`

Limit: 5/min. This is a legacy endpoint that the current UI doesn't use.

Estimates vitals on the server from a face video uploaded through `/api/upload/presign`. It can take several minutes, so set a long client timeout.

Request

```
{ "key": "<video key from presign>", "content_type": "video/webm" }
```

AI assistant

`POST /api/chat`

Limit: 20/min

The assistant can see the user's food history, nutrition summaries, health profile and vitals scans. It uses them to answer questions and to point out concerning readings.

Request

```
{
  "message": "How was my protein this week?",
  "history": [
    { "role": "user", "content": "Hi" },
    { "role": "assistant", "content": "Hi! How can I help?" }
  ]
}
```

`history` is optional. Send the earlier turns of the conversation, oldest first, so the assistant keeps context.

Response

```
{ "reply": "You averaged 72g of protein a day this week..." }
```

`POST /api/weekly-summary/generate`

Limit: 5/min

Generates a new weekly summary for the user and stores it.

```
{ "status": "ok" }
```

`GET /api/weekly-summary`

Limit: 10/min

```
{ "summary": "This week you..." }
```

If no summary exists yet:

```
{ "summary": null, "message": "No weekly summary available yet. Upload food images to generate one." }
```

Public

`GET /api/testimonials`

No auth required. Returns a static list of testimonials for the landing page:

```
[ { "text": "...", "image": "<url>", "name": "...", "role": "..." } ]
```

Typical flows

Scan a meal

1. `POST /api/upload/presign` to get `{url, key}`
2. `PUT` the image to `url`
3. `POST /api/analyze` with `[{key, content_type}]` to get the nutrition result
4. Optionally, `POST /api/meals` to save it as a named meal

Stats dashboard

- `GET /api/nutrition/summary?period=weekly&startDate=…&endDate=…`
- `GET /api/vitals/summary?startDate=…&endDate=…`

Vitals scan

- Run the scan on the device, then `POST /api/facescan/save`